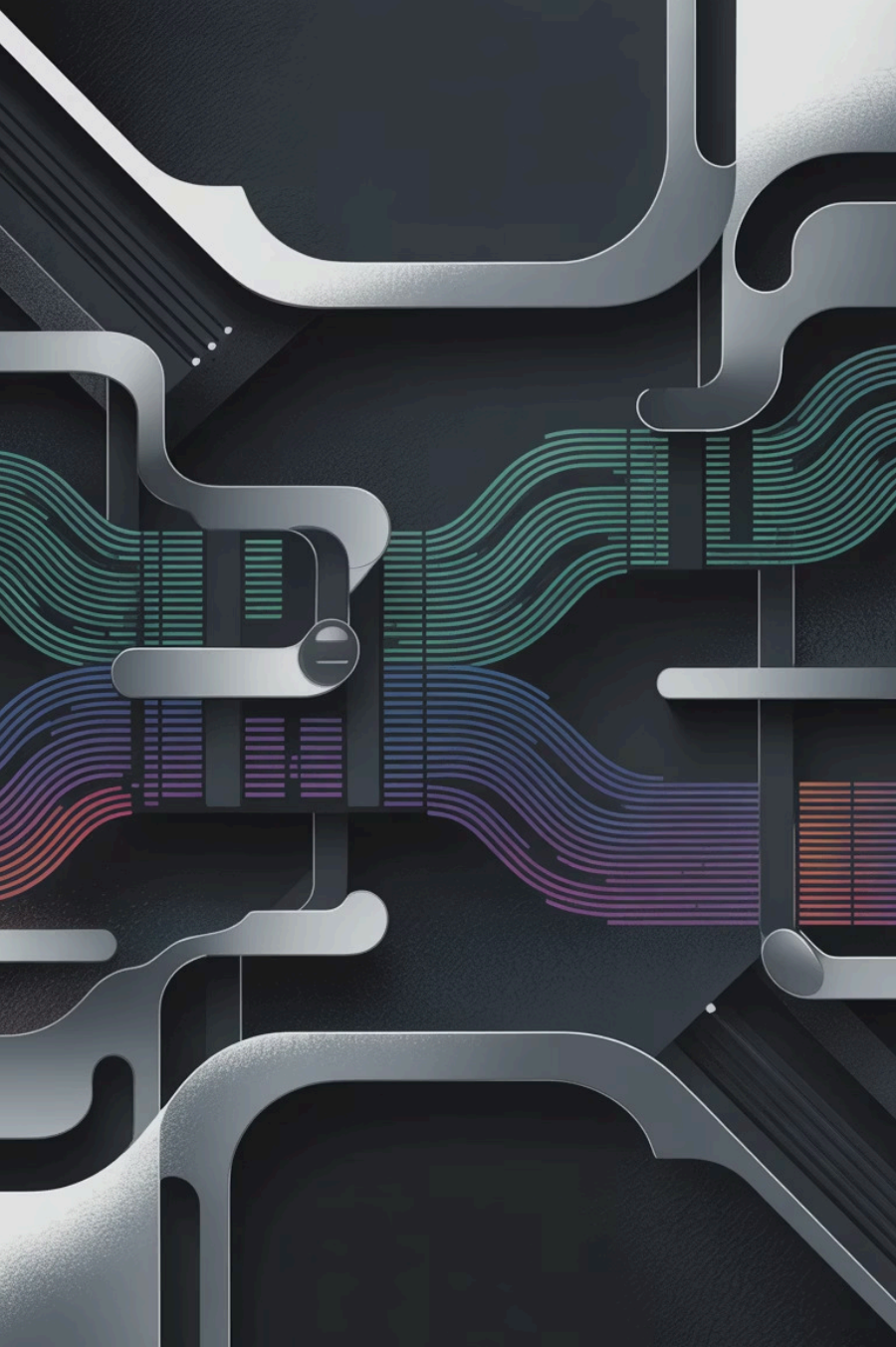




Module 04 - Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Areas of Focus

01

Database Connections

Establish secure connections to SQLite and PostgreSQL databases using Python

02

Parameterized Queries

Write safe, efficient SQL queries with parameter binding

03

Pandas Integration

Load query results directly into DataFrames using `pd.read_sql_query`

04

Chunked Processing

Handle large datasets efficiently with chunked reads

05

Hands-On Lab

Apply these techniques in a practical exercise

Why SQL to Pandas?



The Bridge Between Storage and Analysis

SQL databases store structured data efficiently, but data science workflows happen in Python. The ability to extract, transform, and analyze SQL data in pandas is fundamental for AI engineering pipelines.

- Leverage pandas' powerful data manipulation capabilities
- Prepare datasets for machine learning models
- Integrate with LLM fine-tuning workflows
- Perform exploratory data analysis at scale

Database Options: SQLite vs PostgreSQL

SQLite

File-based, serverless database

- Perfect for development and prototyping
- Zero configuration required
- Portable single-file database
- Built into Python's standard library

Ideal for: Local development, testing, small to medium datasets

PostgreSQL

Production-grade relational database

- Handles concurrent connections
- Supports complex queries and transactions
- Scales to enterprise workloads
- Rich ecosystem of extensions

Ideal for: Production systems, large datasets, multi-user environments

Connecting to SQLite: The Basics

SQLite requires minimal setup. The `sqlite3` module comes with Python's standard library, making it perfect for quick prototyping and local development.

```
import sqlite3
import pandas as pd
from contextlib import closing

# closing() guarantees the connection is closed when the block exits.
with closing(sqlite3.connect("chinook.db")) as conn:
    df = pd.read_sql_query("SELECT * FROM albums LIMIT 5", conn)
    print(df.head())
# connection is now closed
```

📌 **Pro Tip:** Use context managers (with statement) to ensure connections close automatically, even if errors occur.



Connecting to PostgreSQL

PostgreSQL requires the `psycopg` library, which you can install via pip. Connection strings include host, port, database name, and credentials.

```
import os
import pandas as pd
from sqlalchemy import create_engine, URL

# URL.create() safely handles special characters in the password
url = URL.create(
    "postgresql+psycopg2",
    username="analyst",
    password=os.environ["DB_PASSWORD"], # from the environment, not the source
    host="localhost",
    port=5432,
    database="sales_db",
)
engine = create_engine(url)
```

❏ **Security Note:** Never hardcode credentials! Use environment variables, configuration files with restricted permissions, or an external secrets management framework.

The Power of `pd.read_sql_query`

Basic Query Execution

Simple and Powerful

The `pd.read_sql_query()` function executes your SQL query and returns results as a DataFrame in one operation. It handles data type conversions automatically.

Key parameters:

- `sql`: Your SQL query string
- `con`: Database connection object
- `index_col`: Column to use as DataFrame index
- `chunksize`: Number of rows per chunk (optional)

```
import sqlite3
import pandas as pd
from contextlib import closing

with closing(sqlite3.connect("chinook.db")) as conn:
    query = "SELECT * FROM albums LIMIT 10"
    df = pd.read_sql_query(query, conn)
    print(df.head())
    print(f"Shape: {df.shape}")
```

Why Parameterized Queries Matter

SQL Injection Prevention

Parameterized queries protect against SQL injection attacks by separating SQL logic from data values. Never concatenate user input directly into SQL strings.

Query Performance

Database engines can cache and optimize parameterized queries more effectively, leading to better performance when executing similar queries multiple times.

Cleaner Code

Parameterized queries improve code readability and maintainability by clearly separating query structure from dynamic values.

Parameterized Queries: SQLite Syntax

SQLite uses question marks (?) as placeholders for parameters. Pass values as a tuple in the correct order.

```
import sqlite3
import pandas as pd
from contextlib import closing

with closing(sqlite3.connect("chinook.db")) as conn:
    query = """
        SELECT * FROM tracks
        WHERE GenreId = ?
        AND Milliseconds > ?
    """
    params = (1, 200000)
    df = pd.read_sql_query(query, conn, params=params)
    print(f"Found {len(df)} tracks")
```

❏ **Remember:** Parameters must be passed as a tuple or list, even for a single value: `params=(value,)`



Parameterized Queries: PostgreSQL Syntax

When we go through SQLAlchemy we use **named** placeholders with a colon, like `:start_date`, and pass a **dictionary** of values. SQLAlchemy translates those to whatever the underlying driver actually needs.

```
import pandas as pd
# engine created as on the previous slide

query = """
SELECT customer_name, total_amount, order_date
FROM orders
WHERE order_date >= :start_date
      AND total_amount > :min_amount
ORDER BY order_date DESC
"""

params = {"start_date": "2024-01-01", "min_amount": 100.00}
df = pd.read_sql_query(query, engine, params=params)
```


Comparison: Safe vs Unsafe Queries

❌ Unsafe (SQL Injection Risk)

```
# NEVER DO THIS!
user_input = "1 OR 1=1"
query = f"""
SELECT * FROM users
WHERE id = {user_input}
"""

df = pd.read_sql_query(query, conn)

# This could return ALL users!
```

String concatenation or f-strings with user input create severe security vulnerabilities. Attackers can inject malicious SQL code.

✅ Safe (Parameterized)

```
# DO THIS INSTEAD
user_input = "1 OR 1=1"
query = """
SELECT * FROM users
WHERE id = ?
"""

df = pd.read_sql_query(
    query, conn, params=(user_input,)
)

# Only returns user with id="1 OR 1=1"
```

Parameterized queries treat input as data, not executable code. The database safely escapes all special characters.

Handling Large Datasets with Chunked Reads

Why Chunk Your Data?

10M+

Rows in Production DBs

Enterprise databases often contain millions or billions of rows

8GB

Typical RAM Limit

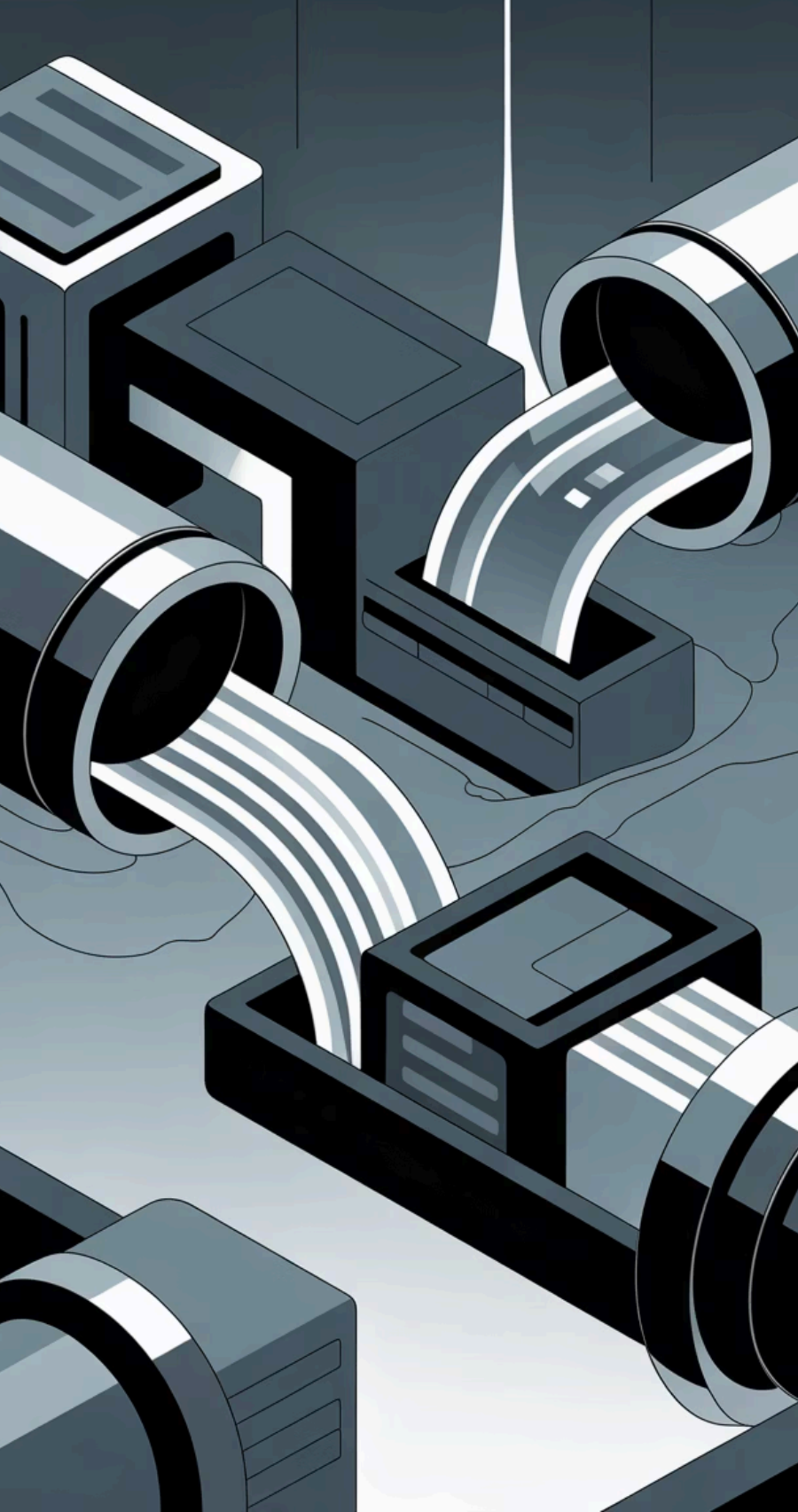
Most development machines have limited memory for loading entire datasets

100K

Optimal Chunk Size

Processing 50K-100K rows at a time balances memory and performance

Loading massive datasets into memory all at once can crash your program or freeze your system. Chunked reading processes data in manageable batches, enabling you to work with datasets larger than available RAM.



Implementing Chunked Reads

Use the `chunksize` parameter to receive an iterator that yields DataFrames of specified size. Process each chunk individually or combine results.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('large_database.db')

query = "SELECT * FROM transactions WHERE year = ?"
params = (2024,)

# Read in chunks of 50,000 rows
chunk_iterator = pd.read_sql_query(
    query,
    conn,
    params=params,
    chunksize=50000
)

# Process each chunk
total_rows = 0
for chunk_df in chunk_iterator:
    # Perform operations on this chunk
    print(f"Processing {len(chunk_df)} rows")

    # Example: filter and aggregate
    filtered = chunk_df[chunk_df['amount'] > 100]
    total_rows += len(filtered)

print(f"Total filtered rows: {total_rows}")
conn.close()
```

Chunked Processing Patterns



Filter and Reduce

Process each chunk to filter rows, keeping only what matches your criteria. Combine filtered chunks into a final dataset.



Aggregate Statistics

Calculate running totals, sums, or counts across chunks. Combine partial results to get final aggregations without loading everything at once.



Write-Through Processing

Transform each chunk and write results to a new database or file. Process entire datasets without ever loading them fully into memory.

Practical Example: Chunked Aggregation

The Challenge

Calculate total sales by category from a 5-million row transaction table without running out of memory.

The Solution

Process data in chunks, aggregating each chunk separately, then combine the partial results into final totals.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('sales.db')

query = """
SELECT category, amount
FROM transactions
WHERE date >= ?
"""

# Initialize aggregation dictionary
totals = {}

# Process in chunks
for chunk in pd.read_sql_query(
    query,
    conn,
    params=('2024-01-01',),
    chunksize=100000
):
    # Aggregate this chunk
    chunk_totals = chunk.groupby('category')['amount'].sum()

    # Add to running totals
    for cat, amt in chunk_totals.items():
        totals[cat] = totals.get(cat, 0) + amt

# Convert to DataFrame
result_df = pd.DataFrame(
    list(totals.items()),
    columns=['category', 'total_sales']
)

print(result_df.sort_values('total_sales', ascending=False))
conn.close()
```

Best Practices Checklist

- **Always use context managers (with) or explicitly close connections**

Close connections explicitly — or wrap them in `contextlib.closing()`

- **Use parameterized queries for any dynamic values**

Essential for security and performance optimization

- **Choose appropriate chunk sizes for large datasets**

Balance memory usage with processing efficiency (typically 50K-100K rows)

- **Store credentials securely in environment variables**

Never commit passwords or API keys to version control

- **Test queries in a database client before coding**

Validate SQL syntax and results before integrating into Python

- **Use LIMIT during development to prototype quickly**

Test your code on small subsets before processing full datasets

Key Takeaways



Database Connections

Master both SQLite (development) and PostgreSQL (production) connection patterns



Parameterized Queries

Always use ? (SQLite) or %s (PostgreSQL) placeholders for secure, optimized queries



pandas Integration

Leverage `pd.read_sql_query()` for seamless SQL-to-DataFrame conversion



Chunked Processing

Use `chunksize` to handle datasets larger than available memory

Next up: Continue building your data engineering skills in the hands-on lab. Apply these techniques to real-world scenarios with the Chinook database!



Thanks